÷ Tractable & Intractable Problem ÷

Computability of Algorithms: Computability studies whether a problem can be solved by an algorithm at all, regardless of time or memory.

"Can a computer ever solve this problem?"

Complexity theory asks: "How fast can we solve the problem?"

Computable problem: A computable problem is one for which:

There exists a finite sequence of steps (algorithm) that always produces the correct answer

Example: Sorting numbers, Finding shortest Path in graph, Searching in a list, multiplying two numbers.

Non-Computable problem: No algorithm can exist that solves the problem for all inputs. Even with infinite time or memory, a computer cannot solve it using an algorithm.

One of the most famous non-computable problems is "The Halting problem."

The Halting problem:

Given: A program P
An input I

Determine whether:

Program P will eventually stop
or
Program P will run forever.

Alan Turing proved that no universal algorithm can determine for every possible program.

Alan Turing and computability: The computability theory was introduced by Alan Turing (1936).

He introduced the concept of a Turing Machine, which is a theoretical model of computation.

A Turing Machine represents:

- Any possible computer program
- Any algorithm

If a problem cannot be solved by a Turing Machine, it is considered non-computable.

Turing Machine Model: A turing Machine consists of:

1. Infinite tape (memory)
2. Read/write Head
3. Set of states
4. Transition rules

Example:- Consider a tape recorder with an endless tape.

The machine:

- Reads a symbol
- Writes a symbol
- Moves left or right
- Changes state

Using this simple model, any modern computer algorithm can be simulated.

Classification of computability problem:- ① Decidable, ② Undecidable

Decidable: If an algorithm exists that always finishes and gives the correct answer.

Example :- Checking if a number is prime, sorting numbers, finding shortest path

Undecidable : If no algorithm can solve it for all input.

Example:- Halting problem, program equivalence problem

Why computability Matters:

1. Which problems are fundamentally impossible to solve
2. Where algorithm design efforts should stop
3. What limits exist in computer science.

Difference between computability and complexity:

	Computability	Complexity
Q:	Can problem be solved?	How fast can it be solved
F:	Existence of algorithm	Efficiency of algorithm
E:	Halting problem	sorting complexity

Real world importance:
1. Sfw verification
2. Artificial intelligence
3. cybersecurity
4. Formal logic